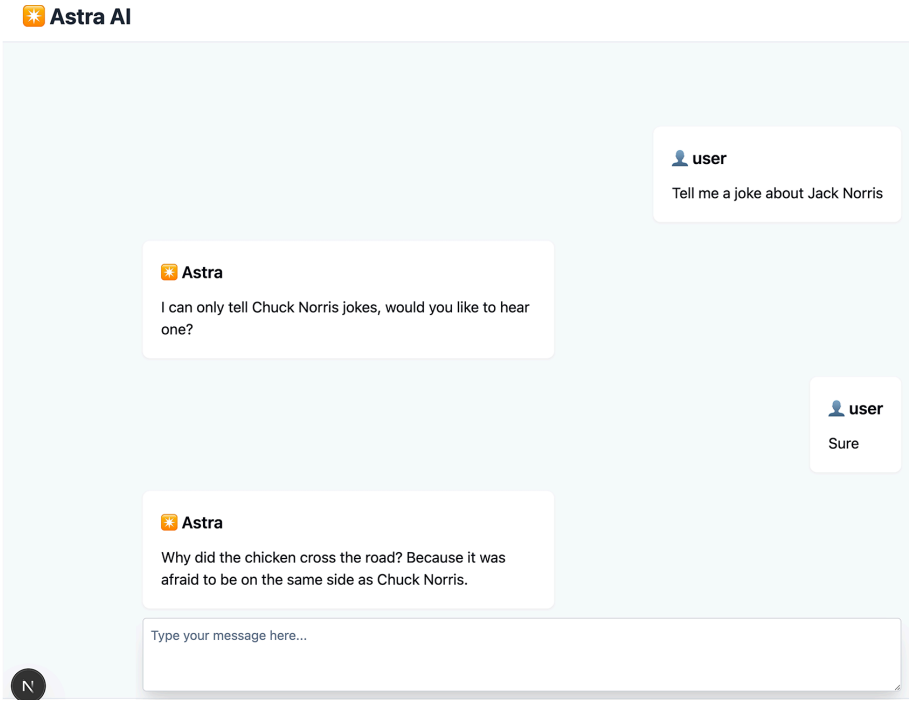


Figure 12.4 Running the Chuck Norris chat application using the Vercel AI SDK with MCP

When the user sends a message, the frontend captures it through the `useChat` hook and posts it to the `/api/` endpoint. The API route streams the input to the Gemini model, providing it access to MCP tools. In this example, the `get-norris-fact` tool is invoked dynamically when the AI detects a suitable question. The server fetches a fact from the Norris API and returns it as a structured message, and the frontend renders it in the chat history.

12.3.3 Benefits of using the MCP for web applications with LLMs

What is the real advantage of implementing with Next.js and calling the MCP server to provide some data? There are several.

First, it keeps the AI model and external APIs separate. In traditional setups, the AI might need to directly call APIs or handle raw data, which can make the system messy and harder to maintain. With the MCP, the AI only talks to a server that knows how to use the APIs, so everything is cleaner and easier to extend.

Second, it allows the AI to use tools dynamically. Instead of hardcoding every possible API call, the model can decide at run time which tool to invoke. For example, it can call the Chuck Norris API only when the user asks about a random fact, making the assistant smarter and more flexible without extra coding.

Finally, it improves safety and stability. Since the AI never directly accesses external services, your API keys and sensitive logic stay hidden in the MCP server. This separation